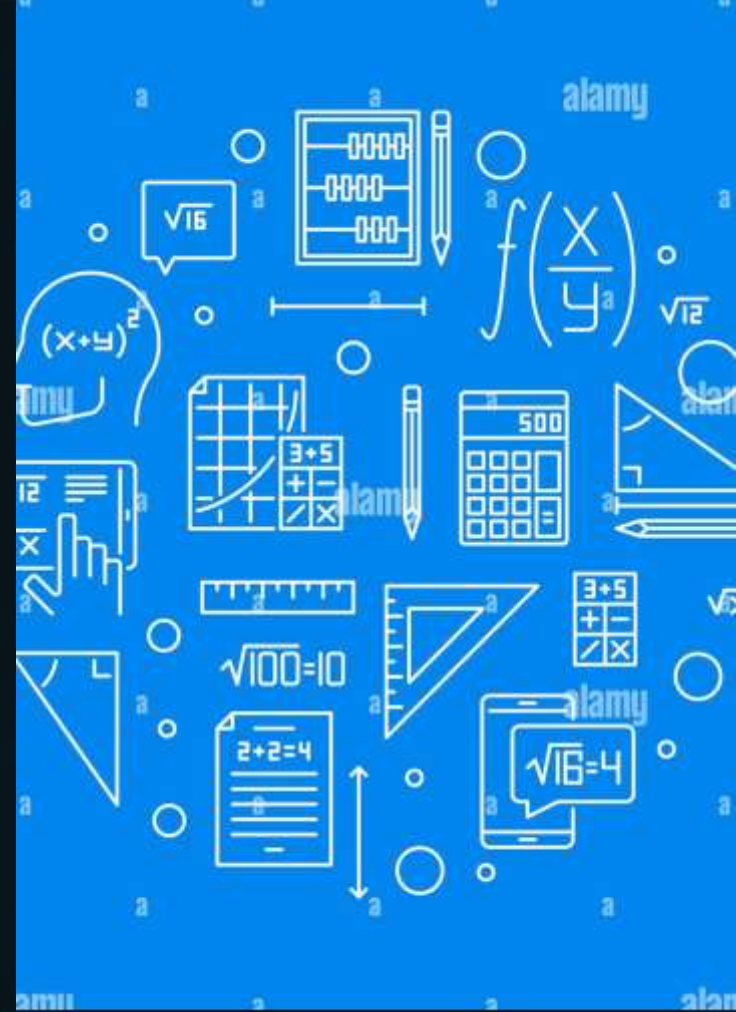


Fundamentos y Jerarquía de Chomsky

Un **lenguaje formal** es un sistema artificial con normas preestablecidas, compuesto de símbolos agrupados en cadenas. Se aplica en lógica, matemáticas, informática y lingüística. El BNF (Backus-Naur Form), creado por John Backus y Peter Naur en los años 60 para ALGOL 60, es la notación estándar para definir la sintaxis de los lenguajes de programación.



Sintaxis Básica de BNF

`::=`

"Se define como". Separa el elemento definido de sus componentes.

`< >`

Encierran los **símbolos no terminales** (variables estructurales).

`" "`

Los **símbolos terminales** (caracteres reales) se escriben entre comillas o de forma literal.

`|`

Significa "o" (alternativa). Separa opciones válidas de una misma regla.

Ejemplo Práctico: Número Entero

Reglas en BNF

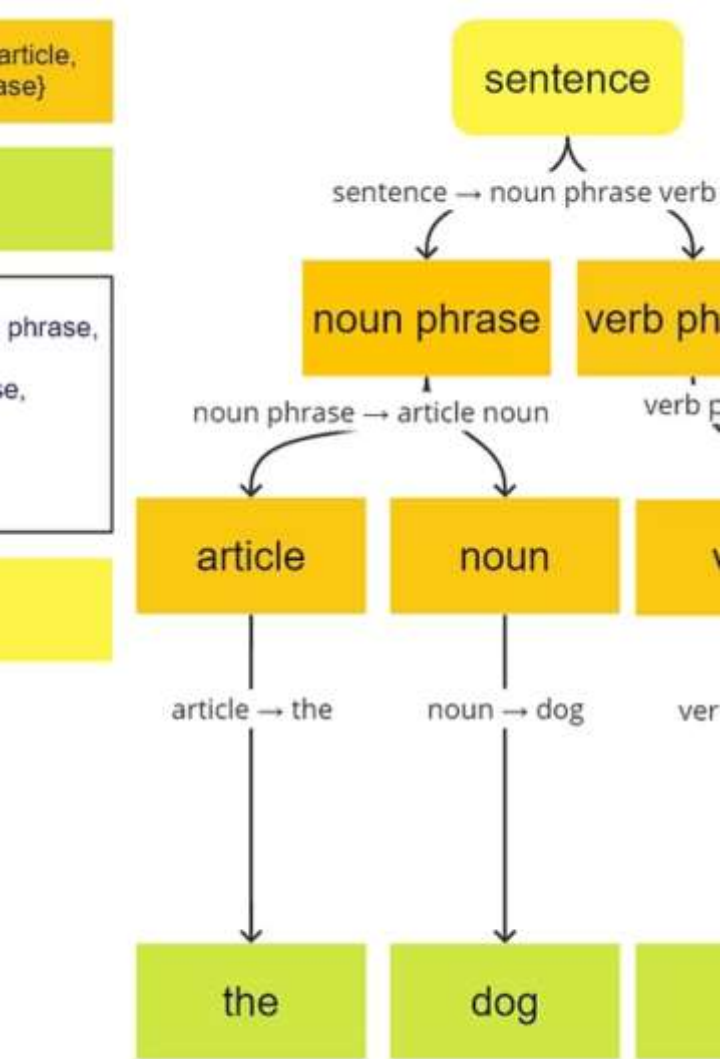
```
<entero> ::= <signo> <digito_no_cero> <resto_digitos> |  
<digito_no_cero> | "0"  
  
<signo> ::= "+" | "-"  
  
<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7"  
| "8" | "9"
```

Interpretación

Un **<entero>** puede ser un signo seguido de un dígito distinto de cero seguido de más dígitos, solo un dígito no cero, o simplemente "0".

Un **<signo>** es el literal "+" o "-".

Los **<dígitos>** son los diez caracteres numéricos del 0 al 9.



Los Cuatro Componentes de una Gramática Formal

Símbolos Terminales (Σ)

Elementos básicos del alfabeto: los caracteres reales que escribe el usuario y reconoce el analizador léxico. Ej: 0-9, +, -.

Símbolos No Terminales (N)

Variables sintácticas intermedias encerradas en < >. Indican qué estructura va en ese lugar, pero desaparecen al final. Ej: <EXPRESIÓN>, <NÚMERO>.

Componentes: Axioma y Reglas de Producción

Símbolo Inicial o Axioma (S)

Punto de partida obligatorio del cual se desglosan todas las combinaciones posibles del lenguaje. Ej: <EXPRESIÓN> como objetivo de construir expresiones matemáticas.

Reglas de Producción (P)

Transformaciones que, partiendo del símbolo inicial, reemplazan no terminales por terminales. Tienen la forma $A ::= B$: "Cuando detectes A, reemplázala por B".



Ejemplo Integrado: Los Cuatro Componentes

Definición Formal

$N = \{ \langle \text{EXPRESIÓN} \rangle, \langle \text{NÚMERO} \rangle \}$

$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, - \}$

$S = \langle \text{EXPRESIÓN} \rangle$

P:

$\langle \text{EXPRESIÓN} \rangle ::= \langle \text{NÚMERO} \rangle "+" \langle \text{NÚMERO} \rangle$
 $\quad \quad \quad | \langle \text{NÚMERO} \rangle "-" \langle \text{NÚMERO} \rangle$

$\langle \text{NÚMERO} \rangle ::= "0" | "1" | "2" | "3" | "4"$
 $\quad \quad \quad | "5" | "6" | "7" | "8" | "9"$

Lectura del Ejemplo

Partiendo de $\langle \text{EXPRESIÓN} \rangle$ (axioma), las reglas permiten generar cadenas como 3+7 o 5-2, reemplazando no terminales hasta obtener solo terminales.



Los no terminales $\langle \text{EXPRESIÓN} \rangle$ y $\langle \text{NÚMERO} \rangle$ desaparecen en el resultado final.

Tipo 0 — Libres o Recursivamente Enumerables

Características

- Los lenguajes Tipo 0 son los más generales de la jerarquía.
- Su gramática es **no restringida** y no exige una forma especial en las producciones.
- Permiten describir procesos de cómputo muy amplios, pero son más difíciles de analizar y decidir.
- Se llaman **recursivamente enumerables** porque una máquina puede reconocer sus cadenas válidas.

Ejemplo en BNF

```
<A> <B> <C> ::= "x"  
<B> <C> ::= <C> <B>
```

Tres variables se reducen a un solo terminal, destruyendo información y encogiéndola la cadena.

Para que el compilador pueda aplicar el cambio, tiene que encontrar obligatoriamente a la variable al lado de la variable <C>. Sino, la regla no se dispara.

Tipo 1 — Dependientes del Contexto

Características

- El símbolo no terminal se reemplaza **solo en ciertos contextos**
- La longitud no se reduce: lado izquierdo $\leq$ lado derecho
- Reconocidos por **Autómata Linealmente Acotado**
- No se usan en lenguajes comerciales: algoritmos muy lentos

Ejemplo en BNF

```
<S> ::= a <A> b
```

```
a <A> ::= a a c
```

El No Terminal <A> solo se transforma en 'a c' si tiene una 'a' a la izquierda

Ejemplo: Ortografía española

El sonido "n" se transforma en "m" solo ante "p" o "b" (*campo, tambor*). Ante "v", permanece como "n" (*enviar*). Sensibilidad al contexto.

Tipo 2 — Independientes del Contexto

Características

- El lado izquierdo es un **único no terminal**; se sustituye sin importar el contexto
- Reconocidos por **Autómata con Pila**
- Base del **100% de los lenguajes de programación** comerciales (Python, Java, C++, Go)
- Es el **analizador sintáctico** del compilador: verifica el orden de tokens según reglas BNF

Ejemplo BNF: Paréntesis balanceados

```
<S> ::= "(" <S> ")" | "a"  
; Genera (a), ((a)), (((a)))...
```

"Un ejemplo que los lenguajes Tipo 3 (regulares) no pueden resolver es el de los paréntesis balanceados (es decir, que por cada paréntesis que abras, se cierre uno obligatoriamente, permitiendo anidamientos infinitos como (((())))"

Tipo 3 — Lenguajes Regulares

Características

- Lado derecho: un terminal solo, o un terminal + un no terminal
- Sin anidamientos infinitos ni memoria de conteo
- Reconocidos por **Autómata Finito (AFD/AFND)**
- Corresponden al **analizador léxico**: lee carácter a carácter y agrupa tokens

Ejemplo: Placa de coche

Un sistema lee **ABC1234** (3 letras + 4 números) de forma secuencial. Si el orden calza, la acepta. Sin estructuras anidadas.

Ejemplo BNF: Identificadores

```
<S> ::= "a" <Resto> | "b" <Resto> | "a" | "b"
```

```
<Resto> ::= "a" <Resto> | "b" <Resto> | "1"  
<Resto> | "2" <Resto> | "a" | "b" | "1" | "2"
```

 Tipo 3 es la base del análisis léxico; Tipo 2, del análisis sintáctico. Juntos forman el núcleo de cualquier compilador.

Derivación y modelado(caso práctico:dibujo)



LENGUAJES Y COMPILADORES · UNEG 2026-I

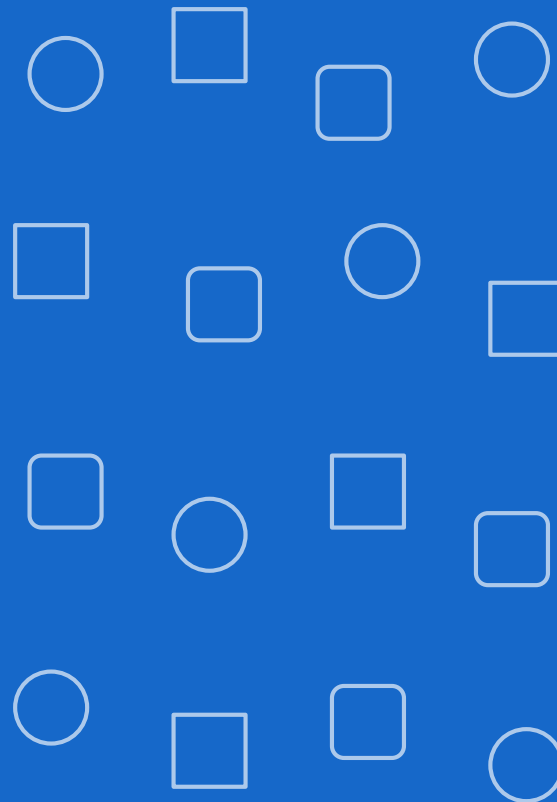
Higiene y Optimización de Gramáticas

Ambigüedad · Recursividad por la izquierda · Factorización por la izquierda

¿Qué es una Gramática Formal?

Una gramática formal $G = (V, \Sigma, P, S)$ es un sistema de reescritura que genera un lenguaje $L(G)$: el conjunto de cadenas obtenibles desde el símbolo inicial S aplicando producciones de P .

Sin embargo, no toda gramática es **apta para construir un compilador**. Algunas presentan patologías que impiden o dificultan el análisis sintáctico determinista.



Tres Patologías de las Gramáticas

Estructuras que impiden construir un analizador sintáctico determinista — y cómo se corrigen.



01 · Ambigüedad

Una misma cadena admite **dos o más árboles** de derivación distintos.

Ejemplo: $E \rightarrow E+E \mid E^*E \mid id$

Solución: estratificar por precedencia (E, T, F).



02 · Recursividad

Izq.

Un no terminal se deriva a sí mismo al inicio
→ **bucle infinito** en parsers LL.

Ejemplo: $E \rightarrow E + T \mid T$

Solución: introducir no terminal E' que acumule la cola.



03 · Factorización

Izq.

Dos producciones comparten **un prefijo común** → el parser no decide con 1 lookahead.

Ejemplo: `if E then S [else S]`

Solución: extraer el prefijo y factorizar el sufijo en S' .

Ambigüedad

Una gramática G es **ambigua** si existe al menos una cadena $w \in L(G)$ que admite dos o más árboles de derivación distintos.

El compilador no puede determinar cuál estructura sintáctica es la correcta, lo que produce código objeto inconsistente.

Gramática ambigua

$G = (\{E\}, \{+, *, id\}, P, E)$

P:

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$



Cadena de prueba

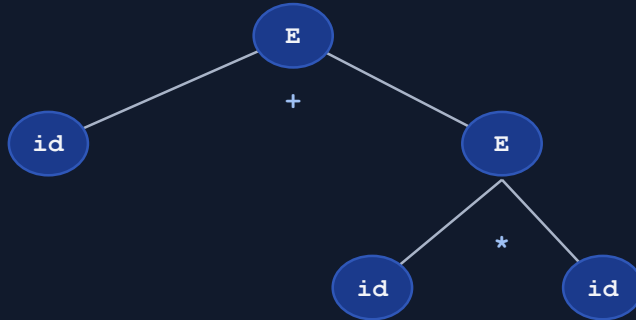
`id + id * id`

Esta cadena pertenece a $L(G)$ y puede derivarse de **dos formas estructuralmente distintas**, cada una con su propio árbol de derivación.

Esto se demuestra en el siguiente slide construyendo ambos árboles y evaluándolos numéricamente.

Dos Árboles para la Misma Cadena

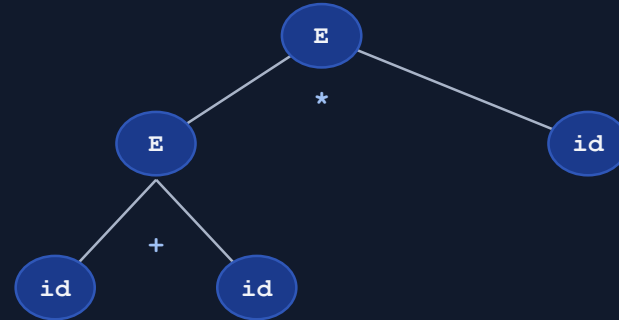
Árbol 1 — suma primero



`id + (id * id) = 6` (con `id = 2`)

La multiplicación queda anidada dentro de la suma.

Árbol 2 — multiplicación primero



`(id + id) * id = 8` (con `id = 2`)

La suma queda anidada dentro de la multiplicación.

Estratificar por Precedencia

Se introduce **un no terminal por cada nivel de precedencia**. Los operadores de menor precedencia quedan en los niveles superiores de la jerarquía.

Gramática G' — no ambigua

$G' = (\{E, T, F\}, \{+, *, (,), id\}, P', E)$

$E \rightarrow E + T \mid T \quad \leftarrow$ precedencia baja (+)

$T \rightarrow T * F \mid F \quad \leftarrow$ precedencia media (*)

$F \rightarrow (E) \mid id \quad \leftarrow$ átomo



Resultado

La cadena $id + id * id$ ahora tiene un **único árbol de derivación**:

$E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T * F \Rightarrow$
 $id + id * id$

La multiplicación siempre tiene mayor precedencia que la suma, sin ambigüedad posible.

Recursividad por la Izquierda

Una gramática tiene recursión izquierda cuando existe un no terminal A tal que $A \Rightarrow^+ A \alpha$ para alguna cadena α .

Gramática problemática

$E \rightarrow E + T$ $\leftarrow E$ aparece primero

$E \rightarrow T$ $\leftarrow$ caso base

$T \rightarrow id$

$\alpha = "+ T"$ $\beta = "T"$



Problema: bucle infinito

Un parser de descenso recursivo implementa cada no terminal como una función:

```
def E():
    E() # llama a E() de nuevo
    match('+')
    T()
```

La primera acción de $E()$ es volver a llamar a $E()$ $\rightarrow$ recursión infinita, nunca consume tokens.

Eliminación Paso a Paso

1

Identificar α

De $E \rightarrow E + T$, extraer $\alpha = "+ T"$ (la cola recursiva).

2

Identificar β

De $E \rightarrow T$, extraer $\beta = "T"$ (la producción base).

3

Crear E'

Nuevo no terminal que acumulará las repeticiones.

4

Reescribir

$$E \rightarrow T E' \qquad E' \rightarrow + T$$

$$E' \mid \varepsilon$$


Verificación – derivación de $id + id + id$

$$E \Rightarrow T E' \Rightarrow id E' \Rightarrow id + T E' \Rightarrow id + id E' \Rightarrow id + id + T E' \Rightarrow id + id + id E' \Rightarrow id + id + id \varepsilon = id + id + id$$

✓

Factorización por la Izquierda

Caso clásico: el **dangling else**. Dos producciones de S comparten el prefijo "if E then S".

Gramática original

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow \text{if } E \text{ then } S$

Prefijo común:

$\text{if } E \text{ then } S$



El problema

Al leer el token "`if`", el parser no sabe cuál producción aplicar.

Necesitaría leer toda la sub-sentencia para saber si hay un `else` — pero eso viola la restricción de lookahead = 1 de LL(1).

Gramática Factorizada

G' — optimizada

$S \rightarrow \text{if } E \text{ then } S \ S'$

$S' \rightarrow \text{else } S \mid \varepsilon$

Tabla de decisión LL(1) para S'

Lookahead	Producción	Significado
else	$S' \rightarrow \text{else } S$	Hay rama alternativa
otro	$S' \rightarrow \varepsilon$	No hay else



Verificación

Con else:

$S \Rightarrow \text{if } E \text{ then } S \ S' \Rightarrow \text{if } E \text{ then } S \ \text{else } S \ S' \Rightarrow \text{if } E \text{ then } S \ \text{else } S \ \varepsilon = \text{if } E \text{ then } S \ \text{else } S \ \checkmark$

Sin else:

$S \Rightarrow \text{if } E \text{ then } S \ S' \Rightarrow \text{if } E \text{ then } S \ \varepsilon = \text{if } E \text{ then } S \ \checkmark$

El parser decide con un solo token de anticipación — la ambigüedad estructural queda resuelta.

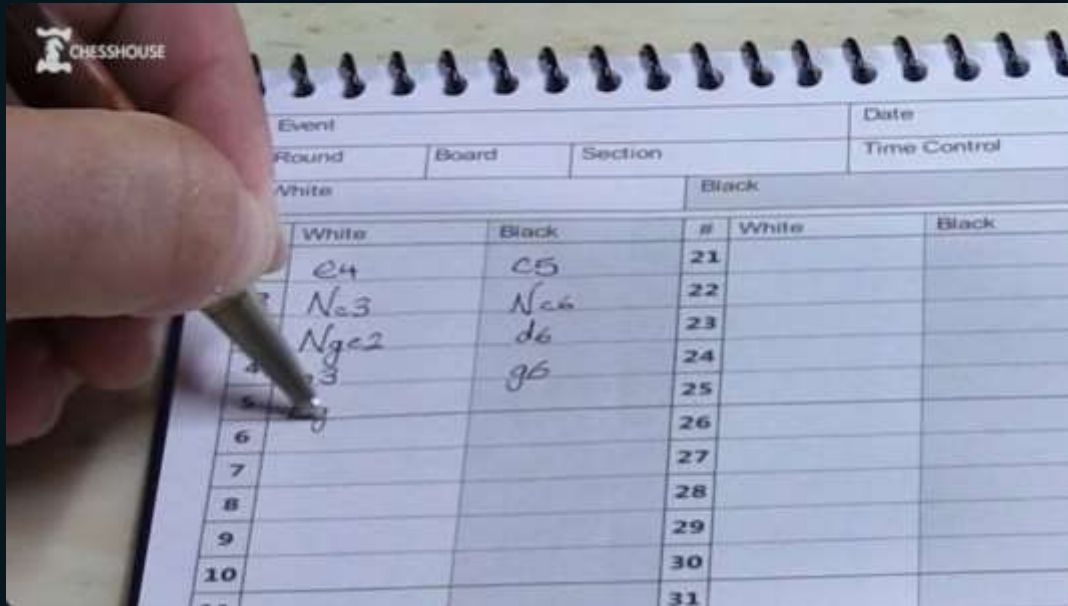
Analizador Sintáctico para Notación PGN

De la Expresión al Autómata — Caso Práctico: Ajedrez

El desarrollo de software para análisis de lenguajes requiere traducir reglas semánticas humanas a estructuras matemáticas procesables en tiempo lineal. Este caso práctico modela un procesador que determina si una cadena corresponde a una jugada válida en ajedrez bajo el estándar PGN.



Notación PGN: Planteamiento Teórico



¿Qué es PGN?

La **Portable Game Notation** es el estándar internacional para registrar partidas de ajedrez en texto plano estructurado.

¿Por qué un AFD?

El PGN completo es un **lenguaje libre de contexto**, pero la sintaxis de un movimiento aislado es un **Lenguaje Regular**. Esto permite usar la herramienta más eficiente: un **Autómata Finito Determinístico (AFD)**.

Ejecución Formal del Autómata

Un AFD ejecuta el reconocimiento mediante la función de transición extendida $\delta^*(q, w)$, donde q es el estado actual y w la cadena restante. Lee un símbolo a la vez, de izquierda a derecha.

✓ Estado de Aceptación

Si al consumir toda la cadena el autómata está en $q_f \in F$, la cadena se declara **Válida**.

✗ Estado Trampa

Si procesa un símbolo sin transición válida, cae en q_e y rechaza la cadena inmediatamente.



Subconjunto Simplificado del PGN

Se define un subconjunto estricto que abarca exclusivamente **movimientos simples de piezas y peones** hacia casillas vacías.

1

Piezas (P)

Letra mayúscula en inglés: K (Rey), Q (Dama), R (Torre), B (Alfil), N (Caballo).

2

Columnas (C)

Letra minúscula que define la coordenada horizontal: {a, b, c, d, e, f, g, h}.

3

Filas (F)

Dígito numérico que define la coordenada vertical: {1, 2, 3, 4, 5, 6, 7, 8}.



Reglas Gramaticales del Subconjunto

Cualquier cadena válida debe cumplir la siguiente estructura secuencial:



Letra pieza

Columna

Fila

La estructura combina un identificador de pieza opcional con coordenadas obligatorias de columna y fila.

Expresión Regular y AFD

Regex Formal

③ `^[KQRBN]?[a-h][1-8]$`

- `^` y `$`: Anclas de inicio y fin de línea, sin caracteres extra.
- `[KQRBN]?`: Cuantificador `?` = opcional (0 o 1 ocurrencia).
- `[a-h]`: Exactamente un carácter de columna.
- `[1-8]`: Exactamente un carácter de fila.



Flujo de Ejecución Paso a Paso

01

Inicialización

Estado inicial **q0**.

02

Ciclo de Lectura

Recorre la cadena índice por índice.

03

Evaluación de Transición

Si el carácter coincide con la matriz de transición, cambia de estado; si no, asigna **qe**.

04

Optimización por Falla (Early Exit)

Si se alcanza **qe**, se interrumpe el ciclo.

05

Veredicto

Si el estado final es **qf**, jugada válida; si no, inválida (cadena corta o con caracteres sobrantes).



Analizador Sintáctico para Notación PGN

De la Expresión al Autómata — Caso Práctico: Ajedrez

El desarrollo de software para análisis de lenguajes requiere traducir reglas semánticas humanas a estructuras matemáticas procesables en tiempo lineal. Este caso práctico modela un procesador que determina si una cadena corresponde a una jugada válida en ajedrez bajo el estándar PGN.



Notación PGN: Planteamiento Teórico

OFFICIAL SCORE SHEET			
EVENT		DATE	
ROUND	BOARD	SECTION	OPENING
WHITE (Name)		BLACK (Name)	
WHITE	BLACK	WHITE	BLACK
1		26	
2		27	
3		28	
4		29	
5		30	
6		31	
7		32	
8		33	
9		34	
10		35	
11		36	
12		37	
13		38	
14		39	
15		40	
16		41	
17		42	
18		43	
19		44	
20		45	
21		46	

¿Qué es PGN?

La **Portable Game Notation** es el estándar internacional para registrar partidas de ajedrez en texto plano estructurado.

¿Por qué un AFD?

El PGN completo es un **lenguaje libre de contexto**, pero la sintaxis de un movimiento aislado es un **Lenguaje Regular**. Esto permite usar la herramienta más eficiente: un **Autómata Finito Determinístico (AFD)**.

Ejecución Formal del Autómata

Un AFD ejecuta el reconocimiento mediante la función de transición extendida $\delta^*(q, w)$, donde q es el estado actual y w la cadena restante. Lee un símbolo a la vez, de izquierda a derecha.

✓ Estado de Aceptación

Si al consumir toda la cadena el autómata está en $q_f \in F$, la cadena se declara **Válida**.

✗ Estado Trampa

Si procesa un símbolo sin transición válida, cae en q_e y rechaza la cadena inmediatamente.

